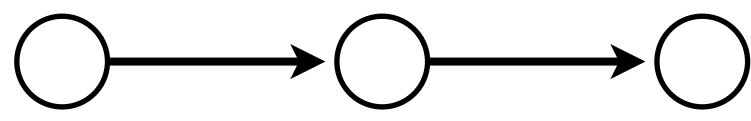


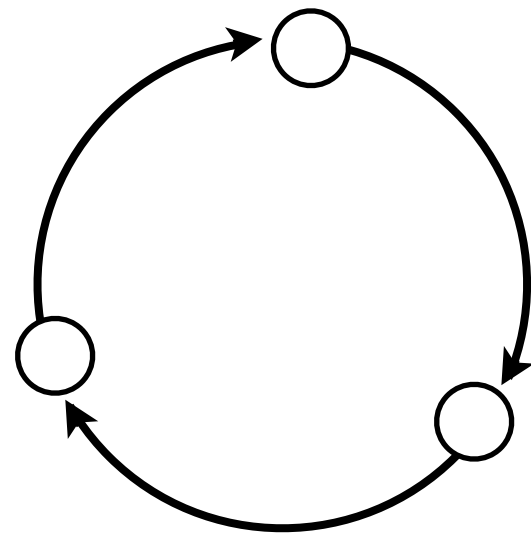
29. Recursion

The Concept of Recursion

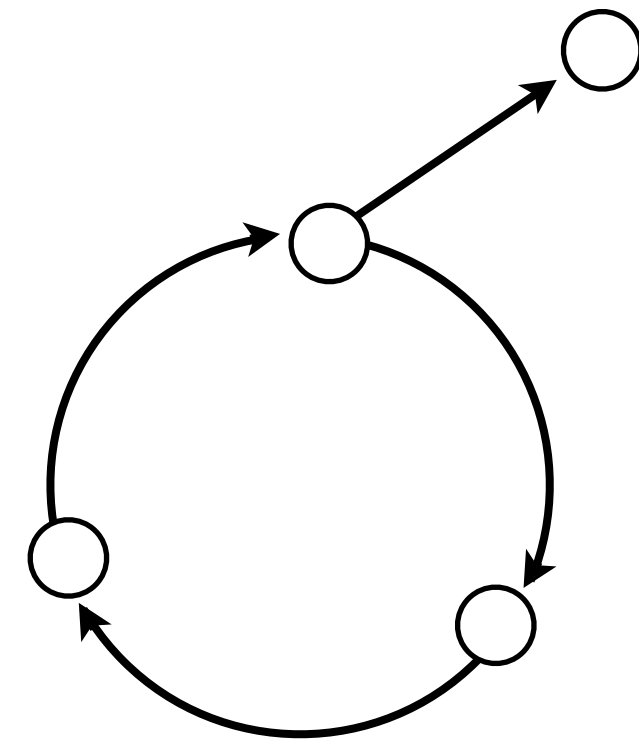
A recursive algorithm repeats some process with slight modifications until some goal is reached. “Recursive” means repeating. Recursive algorithms occur not only in computation, but also in natural processes.



A finite sequence
of states.



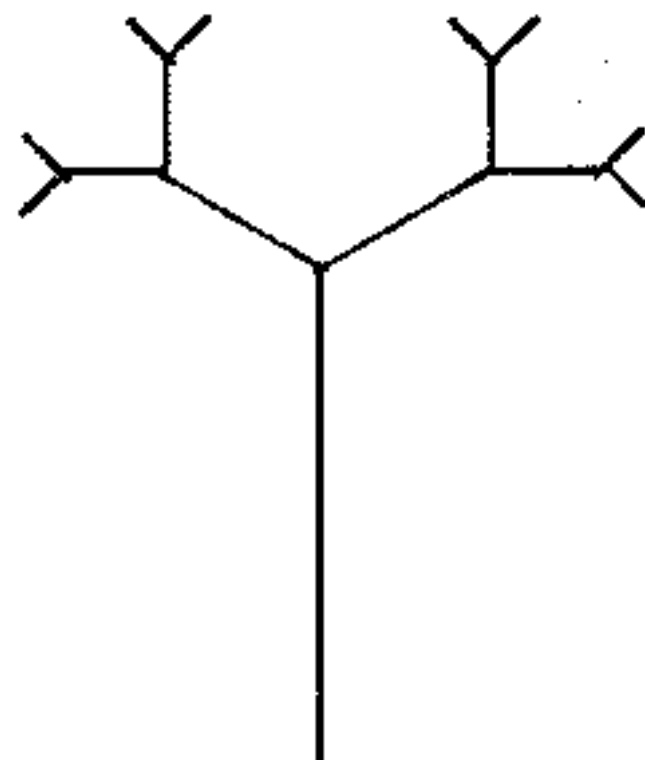
A loop of states which
repeat infinitely.



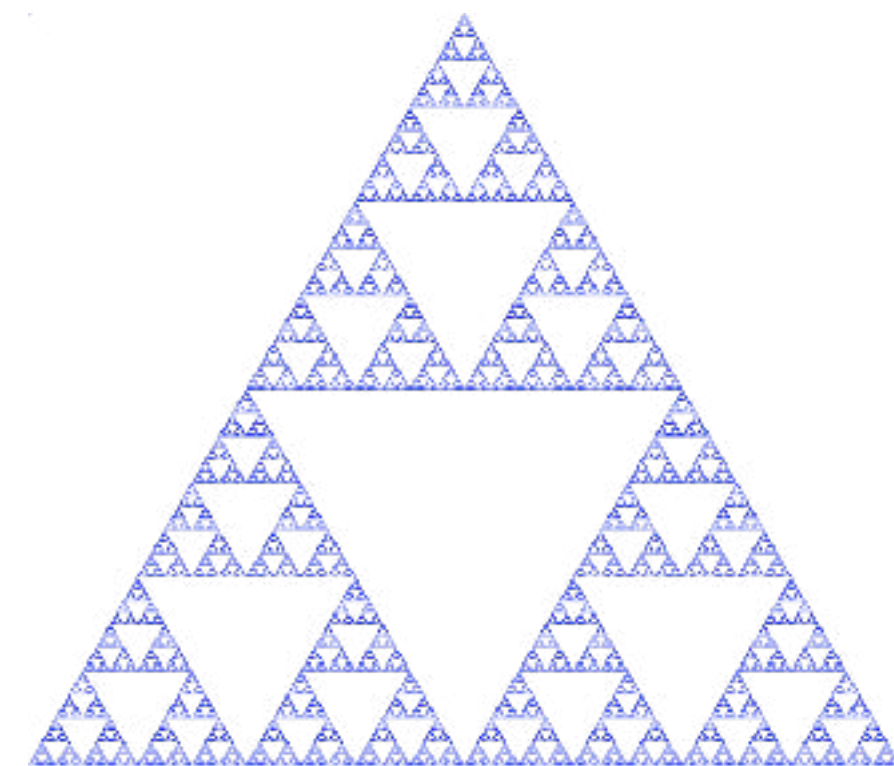
A loop which repeats a
finite number of times until
an exit condition is reached.

Recursive algorithms can be used to generate drawings with elements that repeat, but also change a little bit, after each loop.

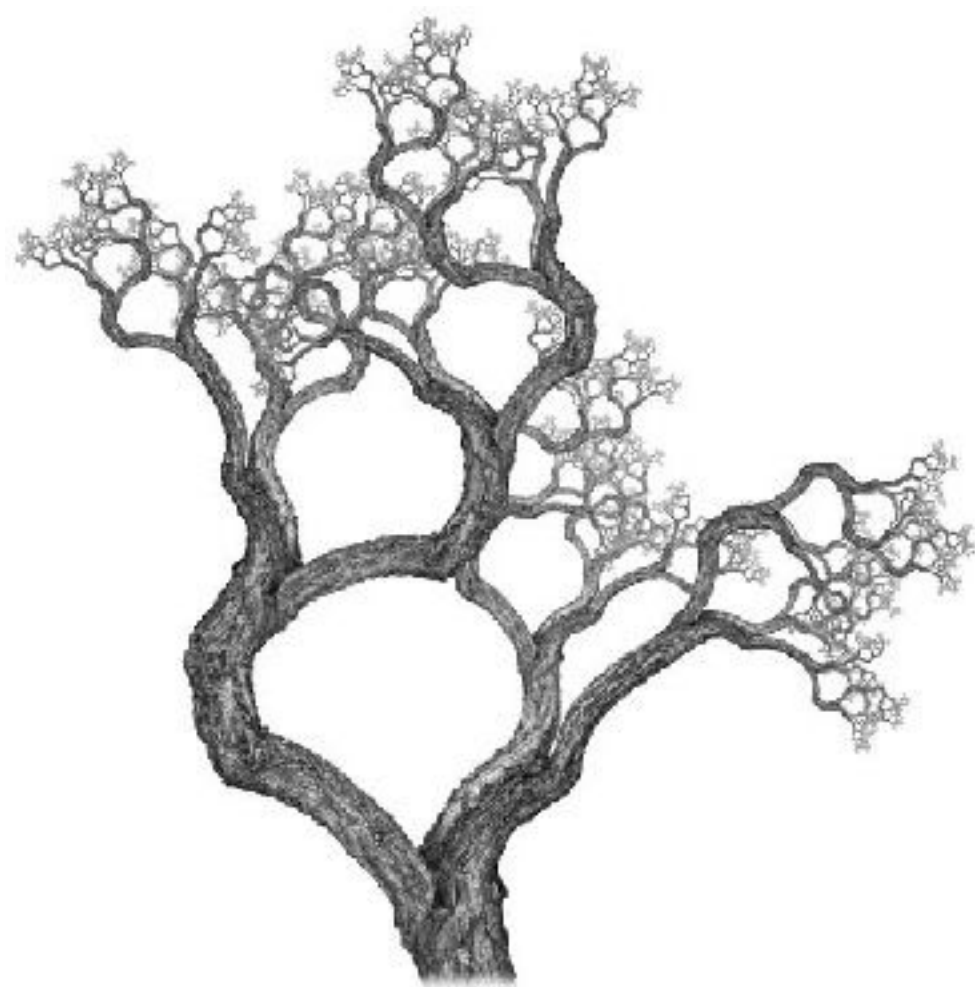
```
TO TREE :SIZE  
FORWARD :SIZE  
IF :SIZE <1 STOP ELSE TWO-  
TREES SIZE/2  
BACK :SIZE  
END
```



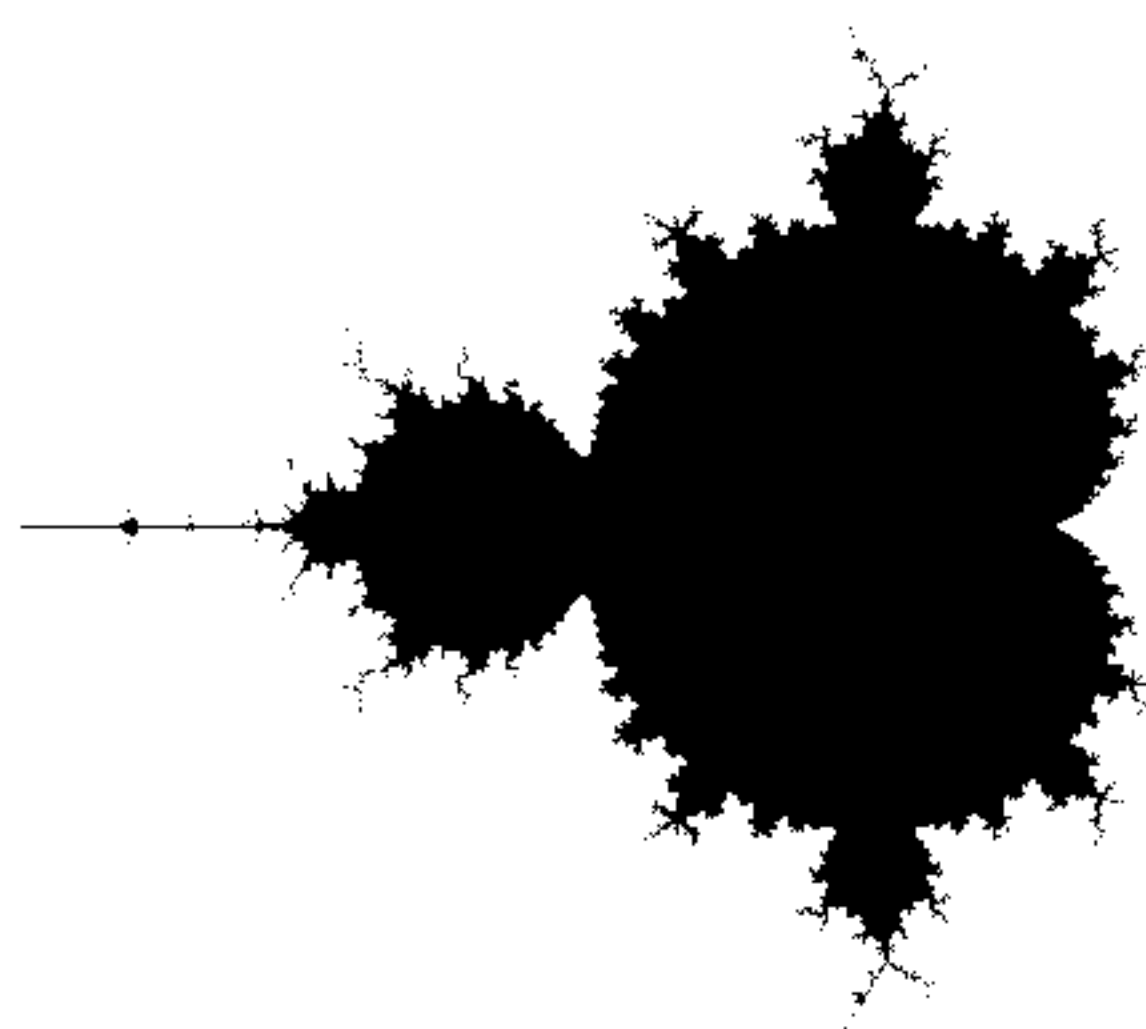
This program written in LOGO describes an
algorithm for drawing simple trees.



How would you generate
Sierpinski's triangle?



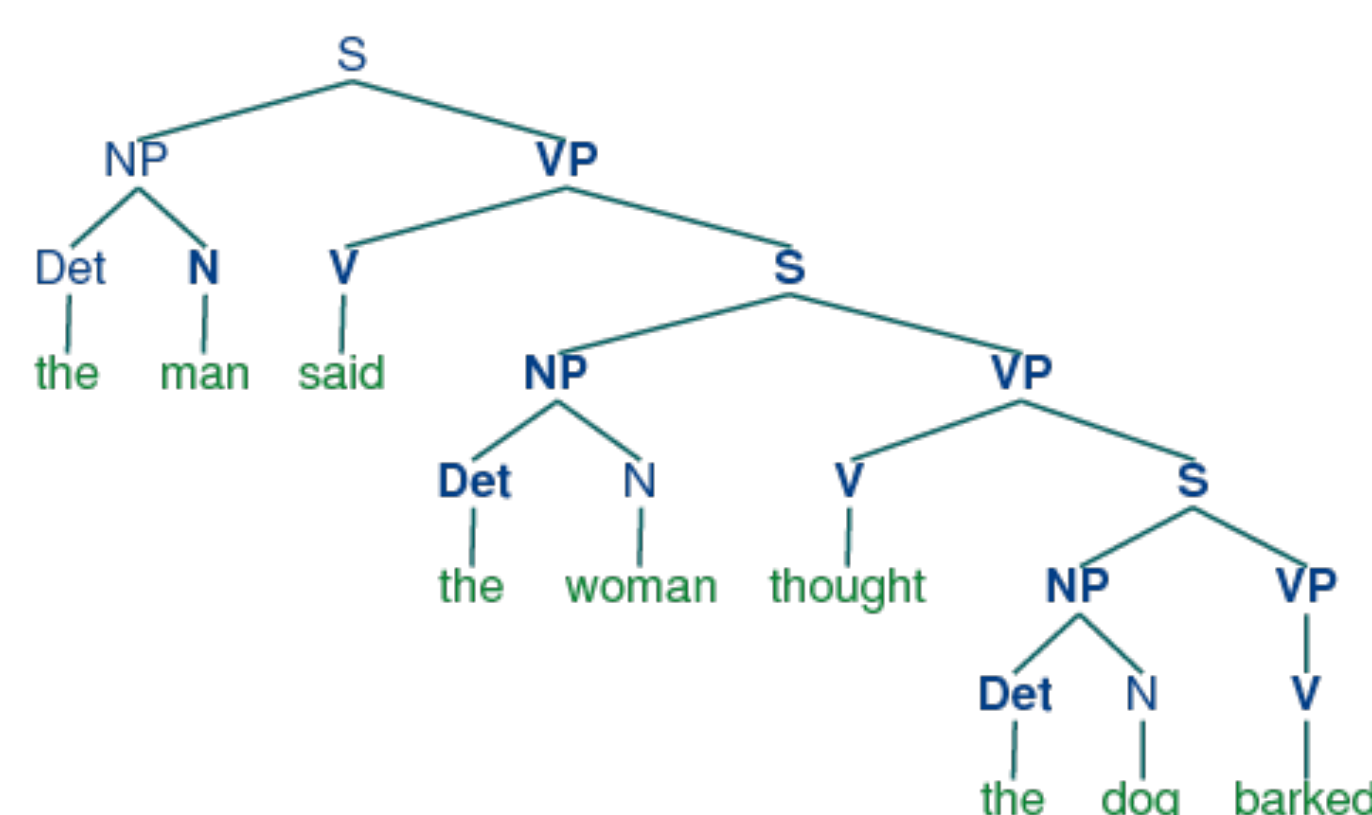
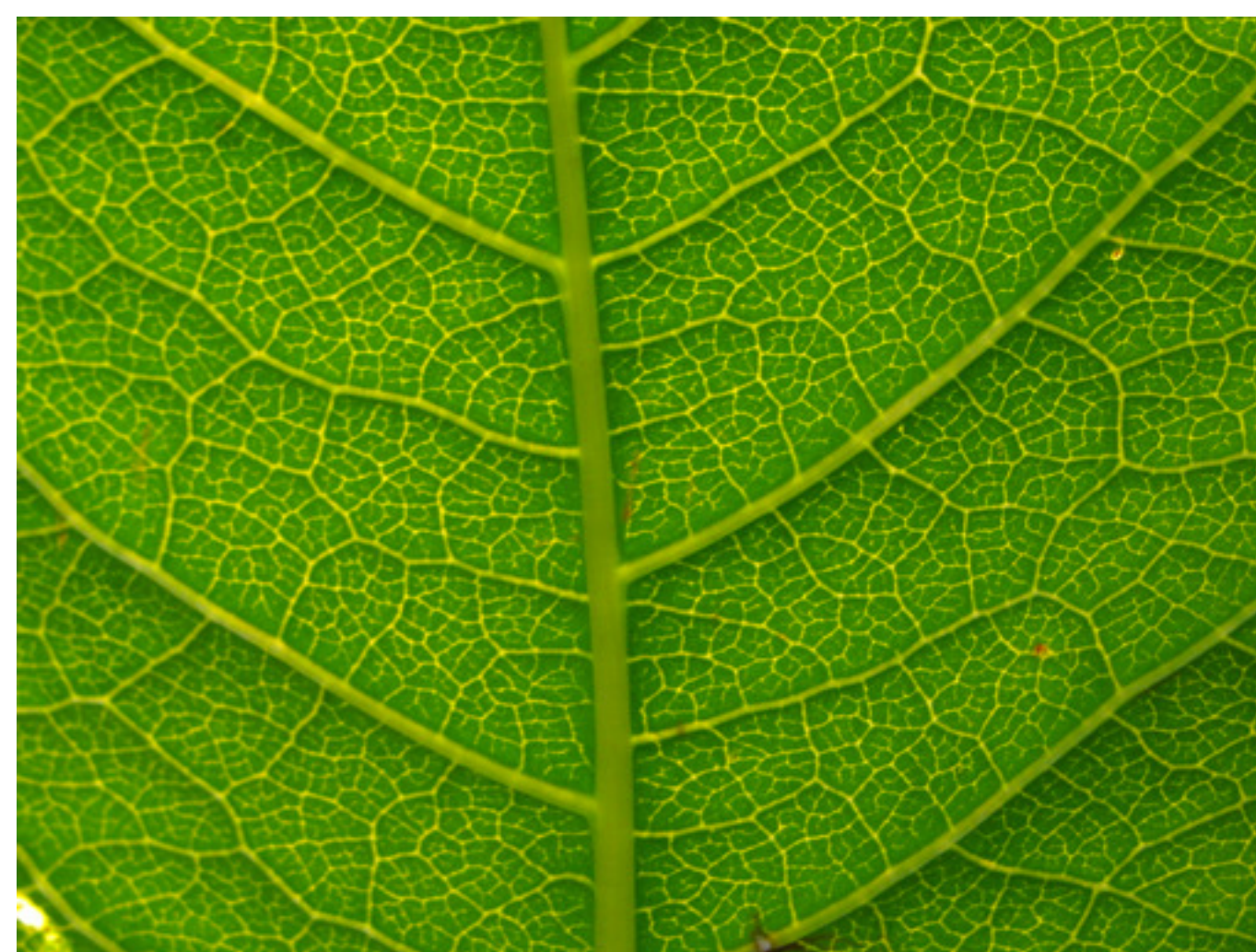
The same technique can be used to draw
more realistic-looking trees...



... or bizarre and complex fractals.

Recursion in Nature

The natural world is filled with recursive structures. Consider a tree: how are the branches generated? How is each leaf on a branch generated? How is each cell is a leaf generated?



Cell growth works by repeating the same kind of operation over and over, with slight modifications made for a single kind of structure (an organ, a leaf), and larger modifications.

Famously, language is also thought to have a recursive structure. Sentences are formed by recursively combining parts of speech.

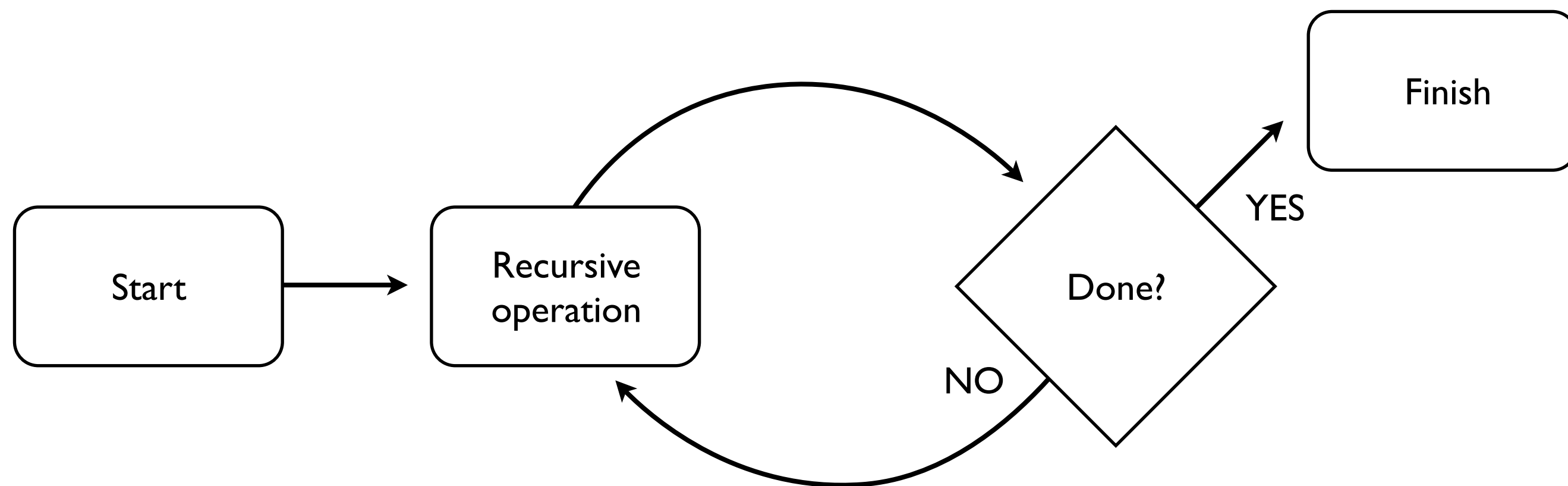
Both cellular growth and sentence structures are driven by recursive operations. But do they have the same root cause?

Perhaps language is recursive *because* biological growth is recursive. Somehow the biological structure of the brain requires it. Perhaps. A more plausible alternative is that both recursive for the same reason. Recursive algorithms are the central mechanism for generating indefinite complexity from a finite base of instructions.

The same pressures which made biology recursive are also what make language recursive, and it is the same pressure which makes computer programs recursive. All face the same kinds of ecological pressure: infinite ends, finite means.

The Architecture of Recursion

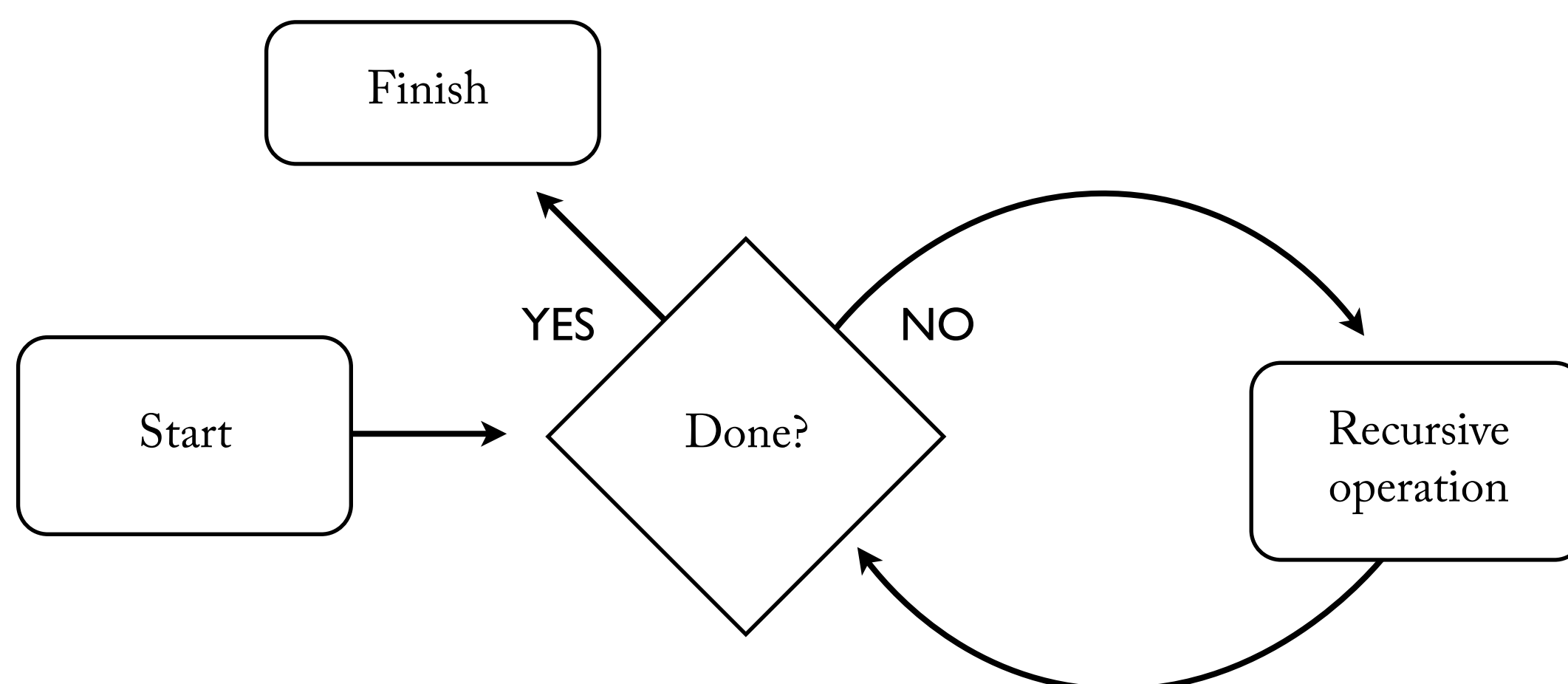
Recursive algorithms have a characteristic architecture. Here is a typical design:



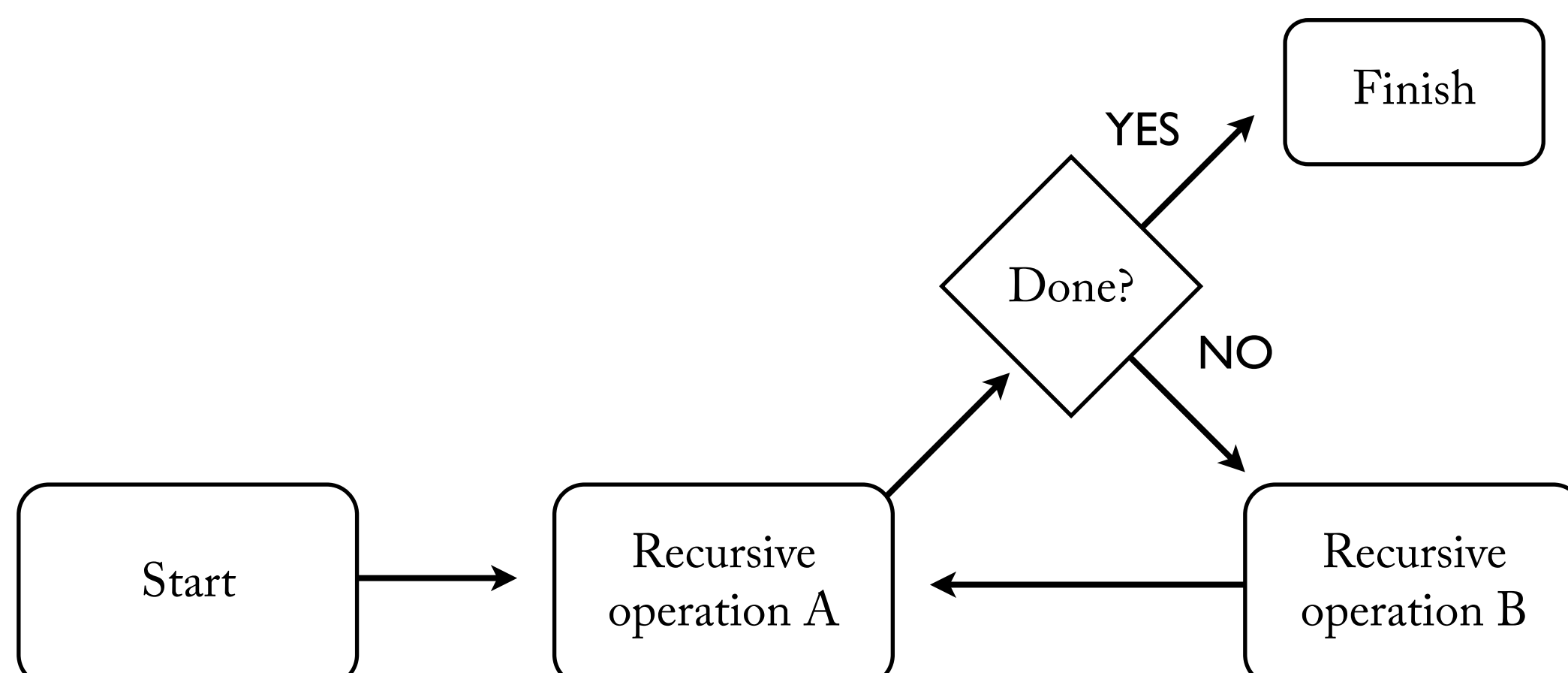
A recursive algorithm has three key elements:

- (1) a **recursive operation** which is a repeated;
- (2) a way to **test** whether the process should be repeated again;
- (3) an **exit operation** when the process is complete.

The three elements may appear in nearly any order. You'll have to find the order best suited for the problem.



Often the recursive operation will itself involve several discrete operations, and these can be re-arranged as well. The underlying conceptual structure is the same.



Note: these diagrams are **flow charts**, a common system for describing algorithms. See the Section “Flow Charts” for more about creating and reading these diagrams.

The goal is to design a TM computes $2x$ in Tally. For present purposes assume $x \geq 1$. Here it is instructive to remember what multiplication really means. Multiplying x by 2 is equivalent to adding 2 to 0, x times. For example:

$$\begin{array}{l} 2 \bullet 5 = 0 + 2 \\ = 2 + 2 \\ = 4 + 2 \\ = 6 + 2 \\ = 8 + 2 \\ = 10 \end{array} \quad \left| \begin{array}{l} \text{5 times} \end{array} \right.$$

BLOCK 1

	0	0	0	0	0	0	0	1	1	1	1	1	0	0
--	---	---	---	---	---	---	---	---	---	---	---	---	---	---

BLOCK 1

	0	0	0	0	0	0	0	1	1	1	1	1	0	0
--	---	---	---	---	---	---	---	---	---	---	---	---	---	---

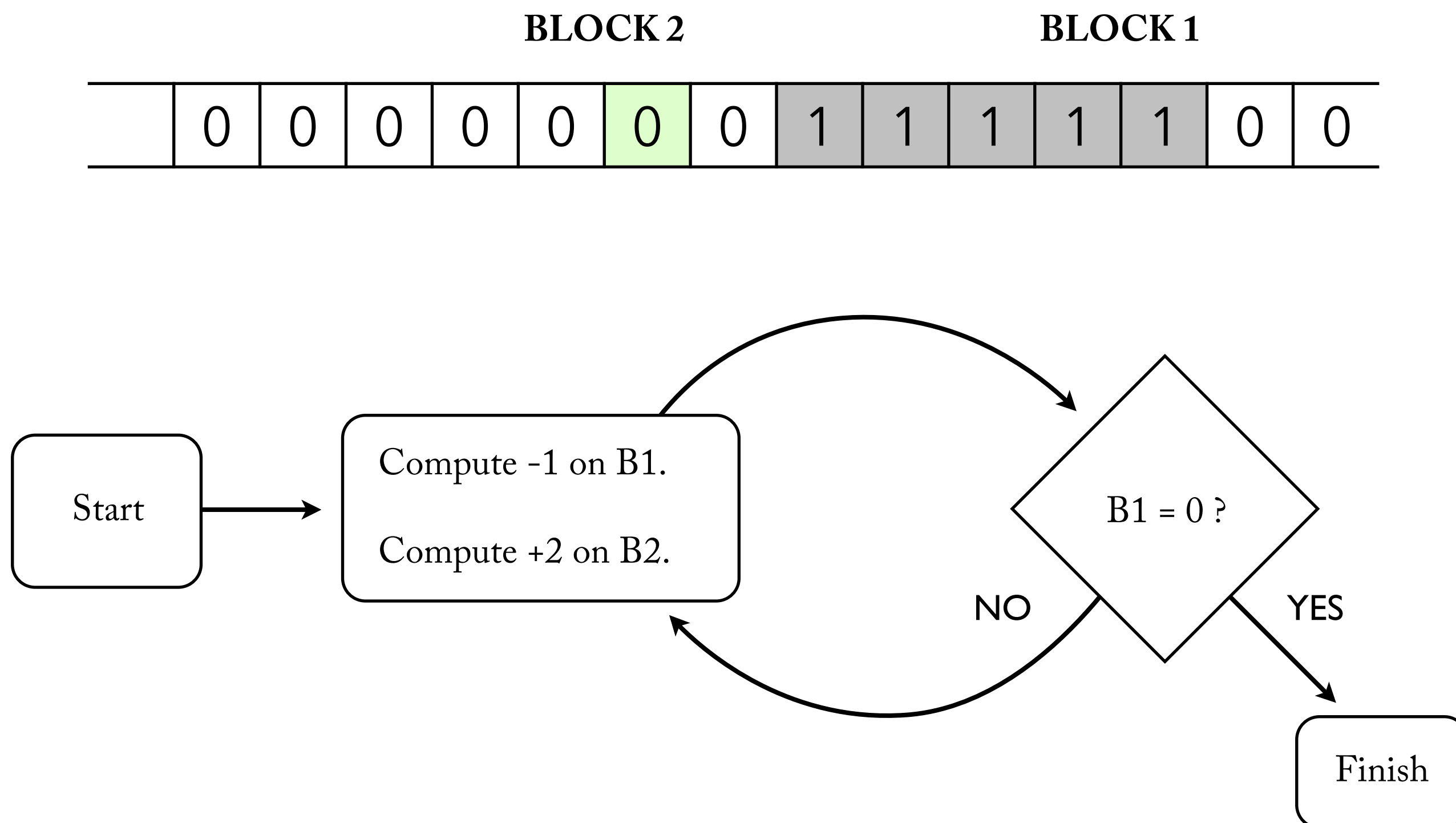
The diagram shows a 15-bit register represented as a horizontal row of 15 boxes. The bits from left to right are: 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0. The 12th bit (the '0' before the final three zeros) is highlighted in light green. Above the 5th bit (the first '1' in the pair '11'), there is a red arrow pointing left labeled $y + 2$. Above the 12th bit (the highlighted '0'), there is a red arrow pointing right labeled $x - 1$.

The diagram shows a 14-bit register represented as a horizontal row of 14 boxes. The bits from left to right are: 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0. The third and fourth boxes (bits 3 and 4) are shaded gray. The eleventh and twelfth boxes (bits 11 and 12) are shaded light green. Above the register, a red arrow labeled $y + 2$ points left towards the third bit. Another red arrow labeled $x - 1$ points left towards the eleventh bit.

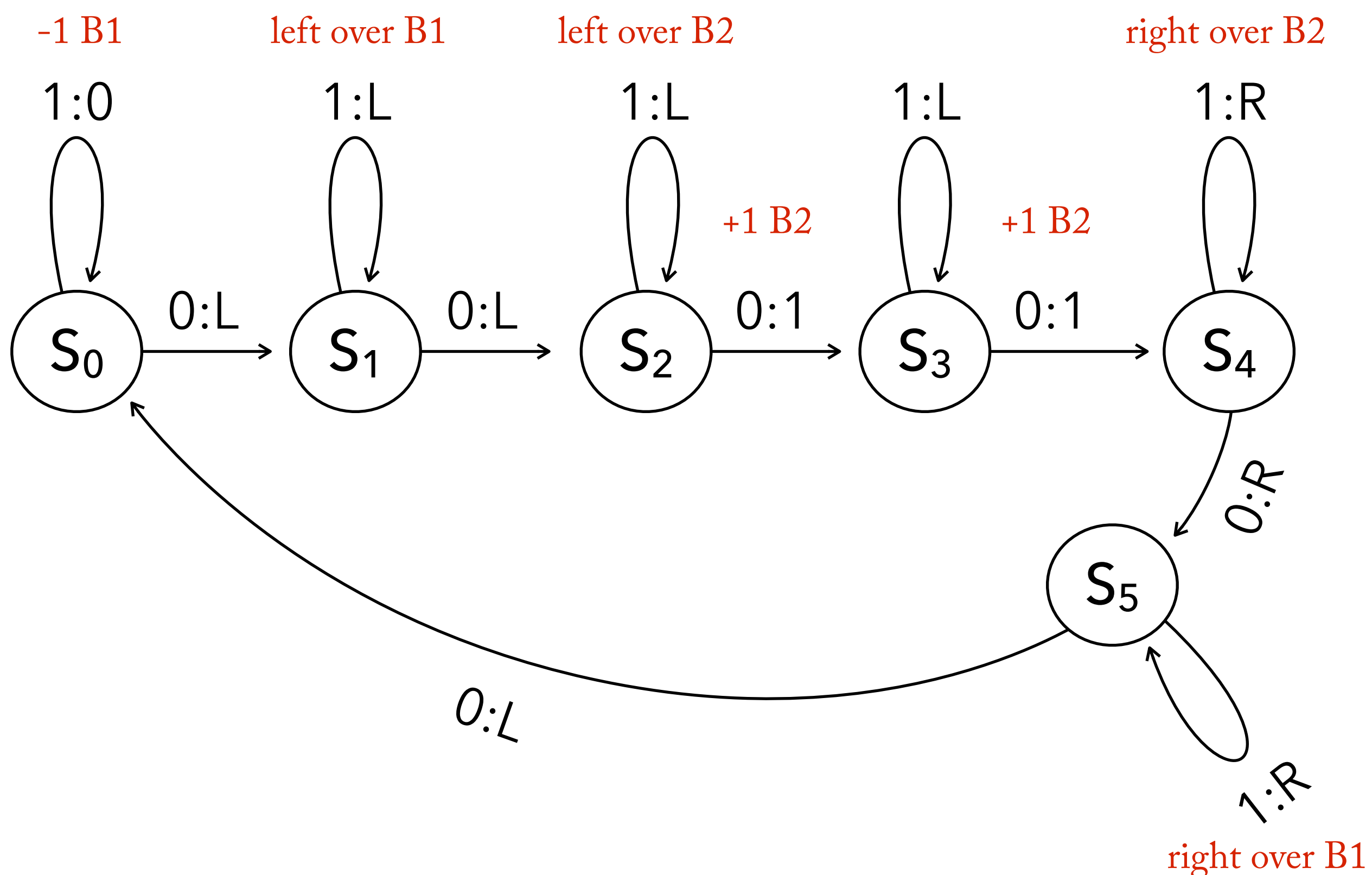
The diagram shows a 14-bit register represented as a horizontal row of 14 boxes. Above the first box is a red arrow pointing left, labeled $y + 2$. Above the 10th box is a red arrow pointing left, labeled $x - 1$. The boxes contain the following values from left to right: 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0. The boxes containing 0 are highlighted in light green.

130

Before drawing up our TM, let us convert our strategy into an explicit flow chart:

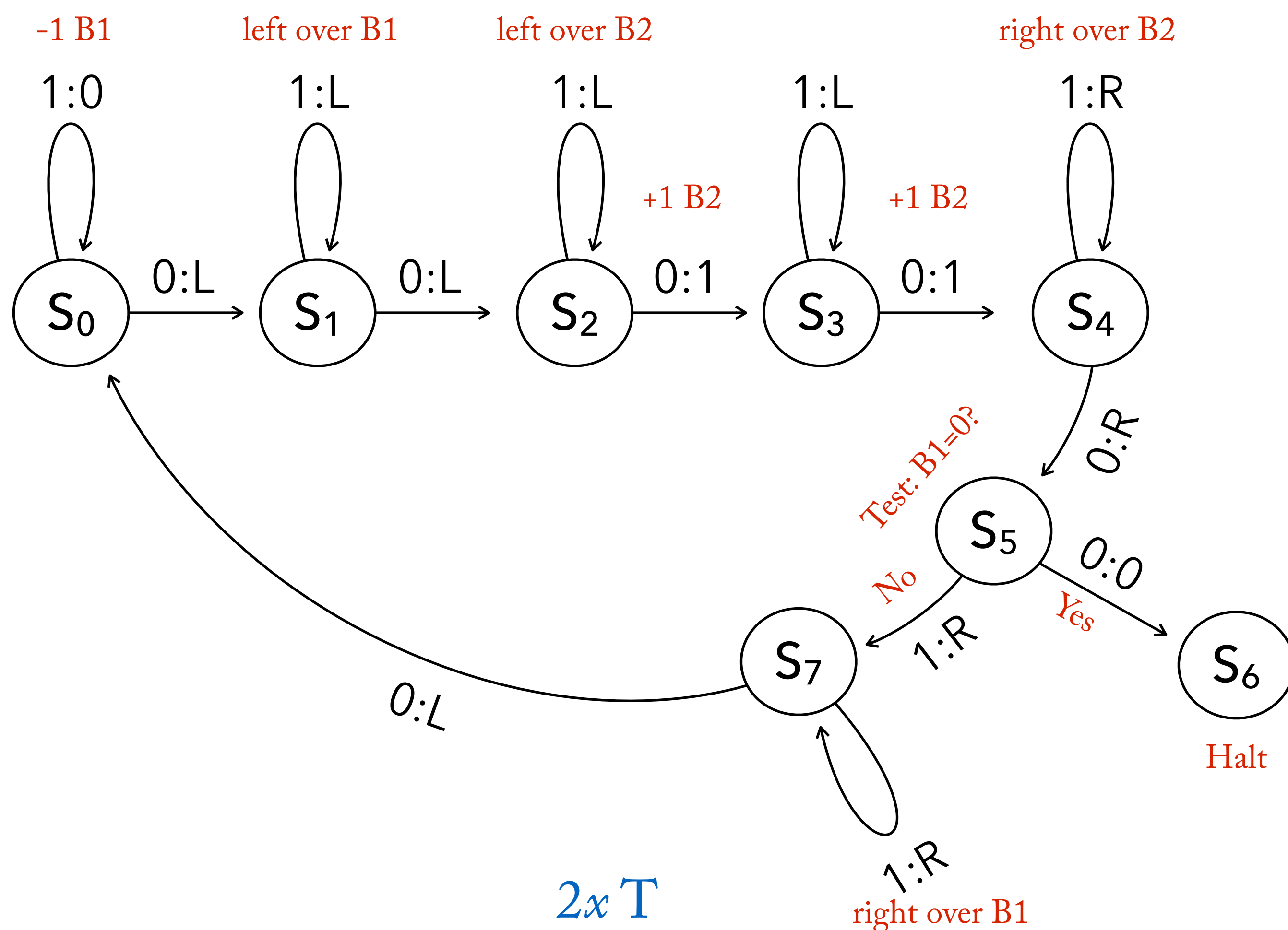


To turn this design into a TM, a good starting place is just to put the critical elements into a loop: (i) -1 from B1, (ii) +1 to B2, (ii) +1 to B2.



The problem with this design is that it contains no **test** or **exit operation**. As a result, it loops indefinitely. Can you predict what will happen to this machine after B1 is empty?

Finally, here is a fully annotated $2x$ Tally TM. It's just like the machine above, but builds in a test to see when B1 is empty, and an explicit exit condition if it is.



Thinking through $x+y$ T

How can we break addition up into a simple, repeating operation? Remember that addition is just successor applied over and over

$$\begin{array}{l} 2 + 3 = 2 + 1 \\ = 3 + 1 \\ = 4 + 1 \\ = 5 \end{array} \quad \left| \quad \begin{array}{l} \text{3 times} \end{array} \right.$$

$$\begin{array}{l} 4 + 5 = 4 + 1 \\ = 5 + 1 \\ = 6 + 1 \\ = 7 + 1 \\ = 8 + 1 \\ = 9 \end{array} \quad \left| \begin{array}{l} \text{5 times} \end{array} \right.$$

The next step is to work through a strategy. Here's a start...

